ECE 3200/5200 Mini - Project SP26

Introduction

In this project, I worked on a supervised multi-class image classification problem. The setup is a bit unique: on the planet Hexagonia, digits come from a base-6 system with values {0, 2, 3, 5, 7, 9}. Each image contains two overlapping handwritten digits, and the goal is to classify which pair appears in the image. Since order doesn't matter, this gives us 15 total classes.

The dataset includes 82,875 labeled training images and 14,625 unlabeled test images. Each image is a 28×28 grayscale image flattened into a 784-dimensional vector. Performance is measured using classification accuracy on a hidden Kaggle test set.

Data Preprocessing

Before training any models, I applied a consistent preprocessing pipeline:

- I normalized all pixel values by dividing by 255.0 to bring them into the range [0,1]. This helps optimization converge faster and more reliably.
- I split the training data into 80% training and 20% validation using stratified sampling, so all 15 classes stayed balanced.
- For the CNN only, I applied data augmentation during training (not validation/test), including small rotations (up to 15°) and affine transformations like translation and scaling. This helped improve generalization.

Methods

Naive Bayes

I first implemented a Gaussian Naive Bayes model. The idea is simple: assume each pixel is independent and model each one with a Gaussian distribution per class.

Training is extremely fast because it's just computing means and variances, no iterative optimization. I tuned the smoothing parameter and got the best validation accuracy at 1e-6. However, performance was low (around 38%). This makes sense because the independence assumption is completely unrealistic for images, where pixels are highly correlated. Also, pixel distributions aren't well-modeled by simple Gaussians.

Softmax Regression

Next, I used softmax regression, which directly models class probabilities using a linear function of the input.

I tuned the regularization parameter and found that C = 0.1 gave the best results. The model was trained using L-BFGS with cross-entropy loss and L2 regularization.

This model performed much better, reaching about 77% accuracy. The training and validation scores were very close, which shows it generalized well. The main limitation is that it's still a linear model, so it can't fully capture the complexity of overlapping digits.

ECE 3200/5200 Mini - Project SP26

Convolutional Neural Network (CNN)

The CNN was by far the best model. I built a residual network with three convolutional blocks in PyTorch. Each block had two convolution layers with BatchNorm and ReLU, followed by pooling and dropout. I also added skip connections to stabilize training and avoid vanishing gradients.

The number of filters increased across layers (64 → 128 → 256), and the final classifier had two fully connected layers.

For optimization, I used Adam with a OneCycle learning rate schedule. I also added gradient clipping to prevent instability and label smoothing to improve generalization.

To push performance further, I trained multiple models with different random seeds and used test-time augmentation (TTA), where each test image is evaluated multiple times with random transformations and the predictions are averaged.

Results

The performance gap between models was very clear:

- Naive Bayes: ~38%
- Softmax Regression: ~77%
- CNN: ~97.7%

The CNN significantly outperformed everything else. The best single model (trained for 80 epochs) actually did slightly better than the ensemble because weaker models pulled the average down.

The key reason the CNN works so well is that it captures spatial structure. Instead of treating pixels independently or linearly, it learns patterns like edges, shapes, and overlaps directly from the data.

Conclusion

This project showed a clear progression in model capability. Simpler models like Naive Bayes and softmax regression struggle because they can't represent spatial relationships well. The CNN, on the other hand, is designed exactly for this type of data and performs much better.

If I had more time, I would push performance even further by training longer (100+ epochs), using a pre-trained architecture like ResNet, and applying stronger augmentation techniques like mixup or cutmix. A stronger ensemble with all models trained longer would also likely improve results.

AI Usage

I used Claude and other tools mainly for setup, debugging, and understanding concepts. This included fixing Python environment issues and implementing the CNN in PyTorch. I also used Google Colab and Kaggle for GPU training and experimentation.